

# Final Project Documentation: CycleGAN

Aeliya Grover | Robert Xing

1. Project Overview
2. Design Choices
3. Results

## 1. Project Overview

For our CMSC 25040 Final Project, we chose to implement an Image-to-Image Translation with CycleGAN to translate tennis balls to baseballs and vice versa.

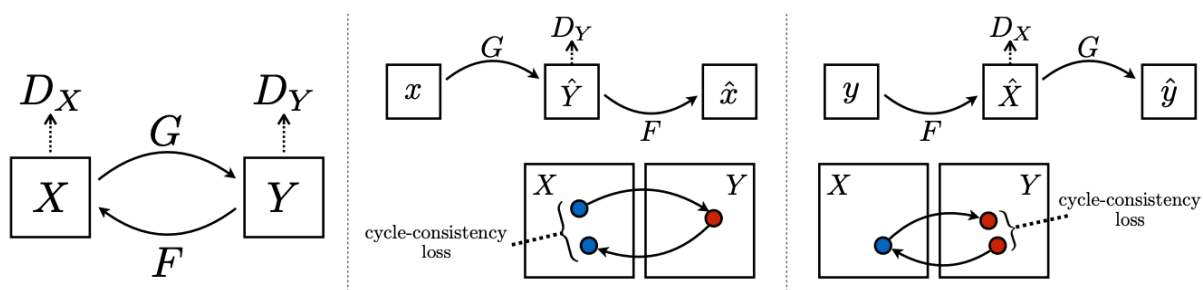
### Data:

The data we used for training and testing can be accessed by downloading it [here](#).

We use a subset of a sports ball dataset from Kaggle, keeping only the images relevant to this particular challenge (tennis balls and baseballs). Original dataset is [here](#).

### Method

We follow a typical CycleGAN architecture, following the original paper, visualized below:



In training, we aim to optimize a loss function that consists of two GAN adversarial losses (one for each generator  $G$  and  $F$ ) plus a cycle consistency loss (weighted by a lambda term), which forms the full objective. The standard formulae for them are below:

### Adversarial loss

$$L_{\text{GAN}}(G, D_Y, X, Y) = E_{y \sim p_{\text{data}}(y)} [\log D_Y(y)] + E_{x \sim p_{\text{data}}(x)} [\log(1 - D_Y(G(x)))]$$

### Cycle-consistency loss:

$$L_{\text{cyc}}(G, F) = E_{x \sim p_{\text{data}}(x)} \left[ \|F(G(x)) - x\|_1 \right] + E_{y \sim p_{\text{data}}(y)} \left[ \|G(F(y)) - y\|_1 \right]$$

Full objective:

$$L(G, F, D_X, D_Y) = L_{\text{GAN}}(G, D_Y, X, Y) + L_{\text{GAN}}(F, D_X, Y, X) + \lambda L_{\text{cyc}}(G, F)$$

Ideal generators:

$$G^*, F^* = \arg \min_{G, F} \max_{D_X, D_Y} L(G, F, D_X, D_Y)$$

$G = X \rightarrow Y$

$F = Y \rightarrow X$

$X = \text{baseball}$

$Y = \text{tennis ball}$

In addition, we also added an identity loss to the full objective, which is essentially the L1 norm of a cyclical translation (e.g.  $\|G(X) - X\|$ , where  $X$  is a real, non-generated image), which is also weighted by its own identity lambda. This further helps enforce cycle consistency by motivating generators to closely match real images.

## 2. Design Choices:

Because we are working with unpaired image-to-image translation, traditional supervised methods won't work as they require paired training data. Since there is no direct mapping between a specific tennis ball image and a baseball image, a CycleGAN model structure is more appropriate.

Data processing:

First, we take the baseball and tennis ball images from the Kaggle set and split them into training and testing. Images are resized to 256x256 and normalized to maintain proper

consistency with training. Example images are shown below.



### CycleGAN architecture:

There are four model networks:

#### Generators:

- G: translates  $X \rightarrow Y$  or takes tennis ball images and turns them into baseball images
- F: translates  $Y \rightarrow X$  which takes baseball images and turns them to tennis ball images
- For the generator structure, we use convolution layers, residual blocks, and upsampling in an encoder-decoder architecture.
- Convolution layers help extract visual features from the images, like edges, shapes, and textures.
- Residual blocks allow the network to modify the image while keeping important information from the original image.
- Upsampling increases the spatial size back to the output image resolution after earlier layers of compression.
- Residual Blocks learn only the change needed to make to the input. This is important because we want to make sure our model keeps the object position, shape, and background layout while changing patterns like texture and color.

#### Discriminators:

- $D_Y$ : distinguishes real baseballs vs generated ones
- $D_X$  distinguishes real tennis balls vs generated ones
- Following a PatchGAN style
- PatchGan evaluates an image as real or fake by looking at small patches of the image.
- As such, the discriminator outputs a feature map of authenticity scores, not a singular value.

### Loss:

Following the outlined loss functions, our model uses:

1. Adversarial loss - Trains the generator to produce translated images that fool the discriminator while the discriminator learns to distinguish real images from generated ones.
2. Cycle Consistency loss - If an image is translated from one domain and back, the reconstructed image should look close to the original. Ensures the preservation of image structure
3. Identity loss - Trains the generator to make minimal changes, helping reduce excess modifications.
4. Total loss - The sum of the adversarial, cycle consistency, and identity loss

#### Process Flow:

1. Generate fake images using generators
2. Update Discriminators to classify real vs fake images
3. Update Generators to fool the discriminators
4. Compute Cycle + identity losses
5. Backpropagate total loss

#### Linear Decay:

Linear decay is used to reduce the learning rate gradually during later epochs. This helps stabilize training and allows the generators and discriminators to converge more smoothly.

#### Scheduler:

A learning rate schedule is used to adjust the optimizer's learning rate during training.

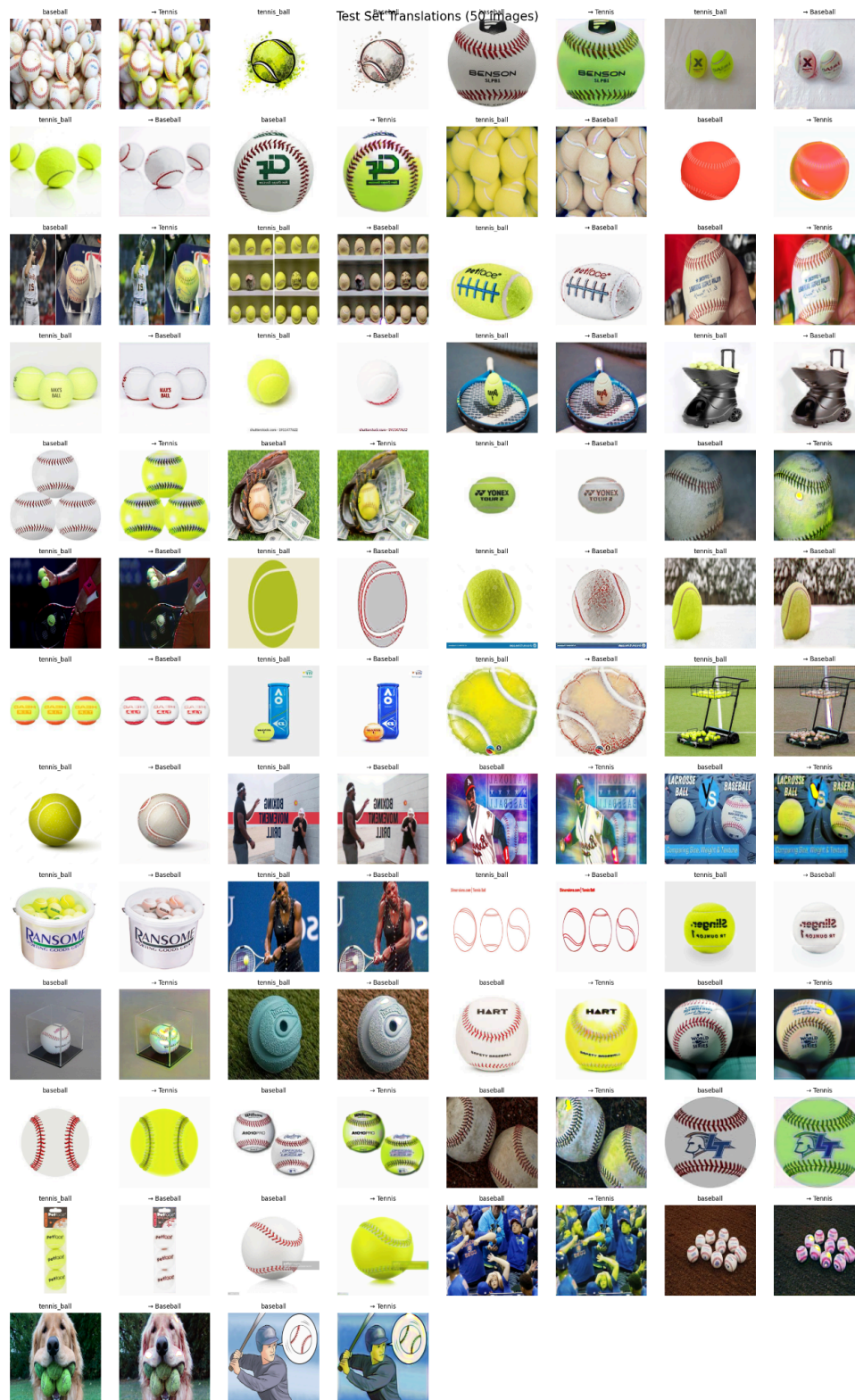
#### Checkpoints:

Checkpoints were saved during training to save snapshots of training to allow the ability to pause and unpause training.

### 3. Results:

Here we show a sample of 50 images from the test set.

Output Notes:



The model is able to preserve background, scene composition, and object placement while modifying mainly color, seam texture, and surface shading. This suggests the generators learned the texture mapping.

When there's a clear yellow outline of a tennis ball, the model can accurately change and modify the texture to a baseball.

However, images with different colored and pattern markings have a slight struggle with the translation.

Additionally, the exchange from a tennis ball to a baseball works significantly better than translating from a baseball to a tennis ball. A likely reason is that the baseball domain is visually simpler and more consistent. The tennis balls in the dataset vary more in color, texture, branding, and context, contributing to a harder target differentiation.

Training loss history:

